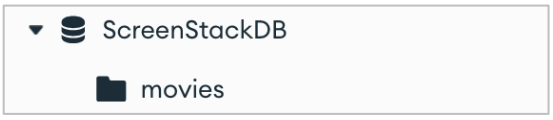
# Prototyping – Übung 6

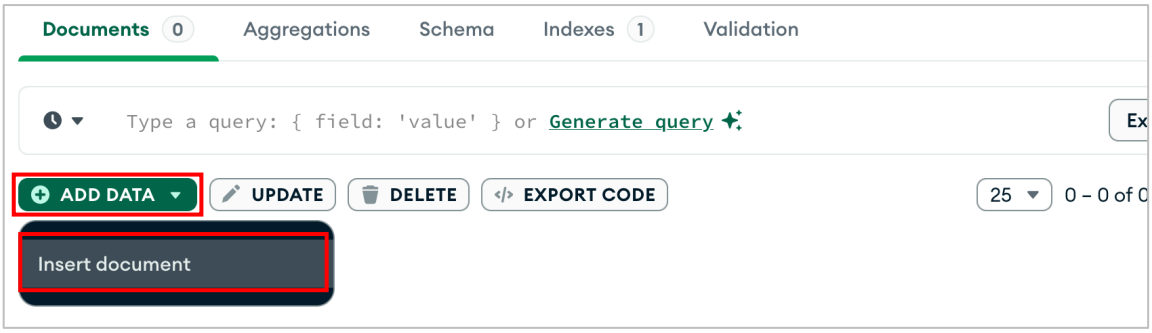
**Themen: SvelteKit, Daten in einer MongoDB verwalten**

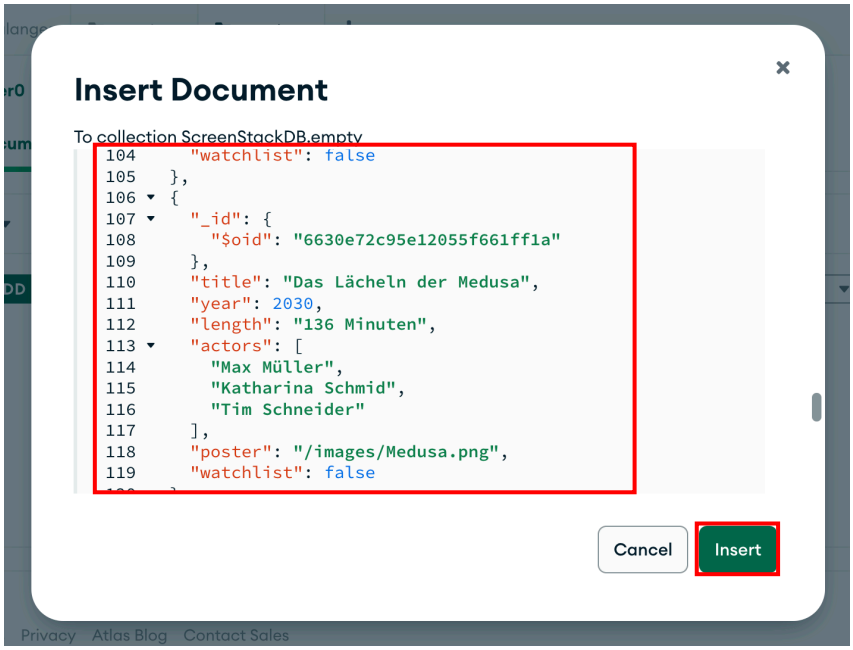
Starte entweder mit der Beispiellösung oder deiner eigenen Lösung der Übung 5.

a) Als erstes benötigst du Demodaten in einer MongoDB. Erstelle dazu mit Compass oder auf https://cloud.mongodb.com eine neue Datenbank *ScreenStackDB* mit der Collection *movies*:

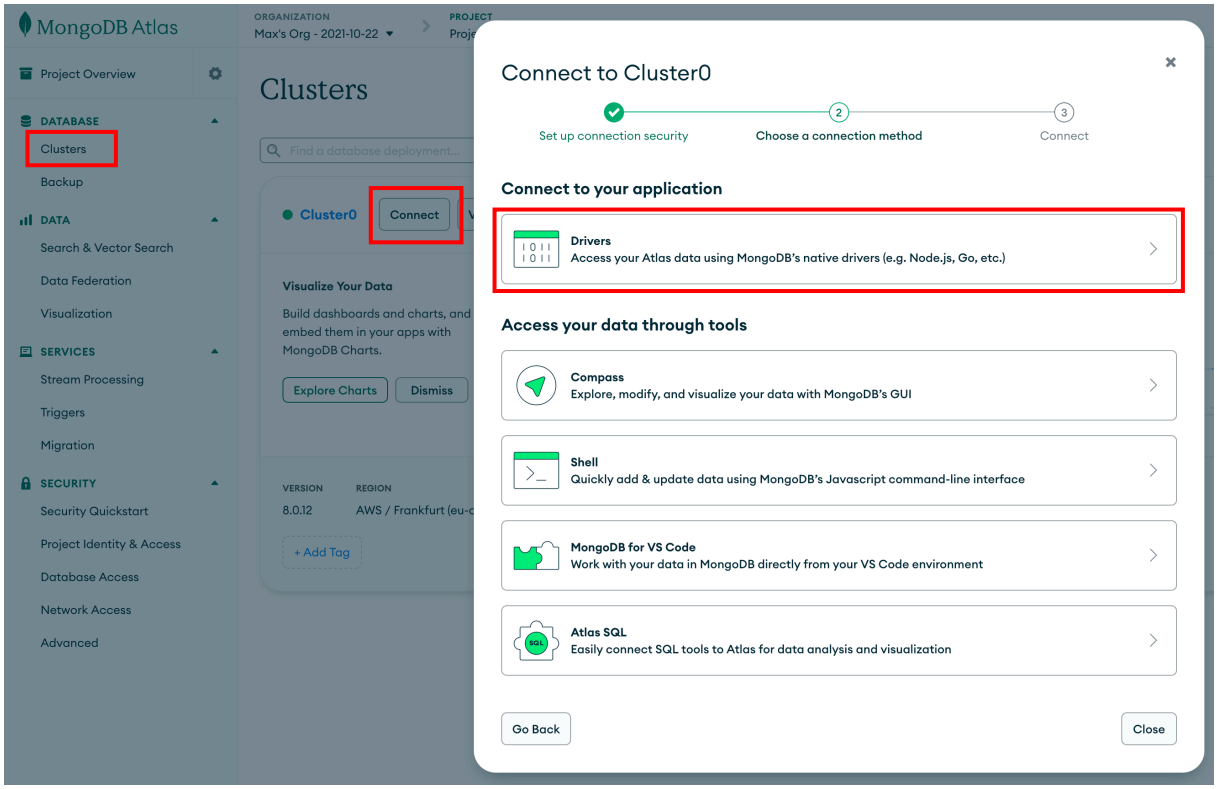


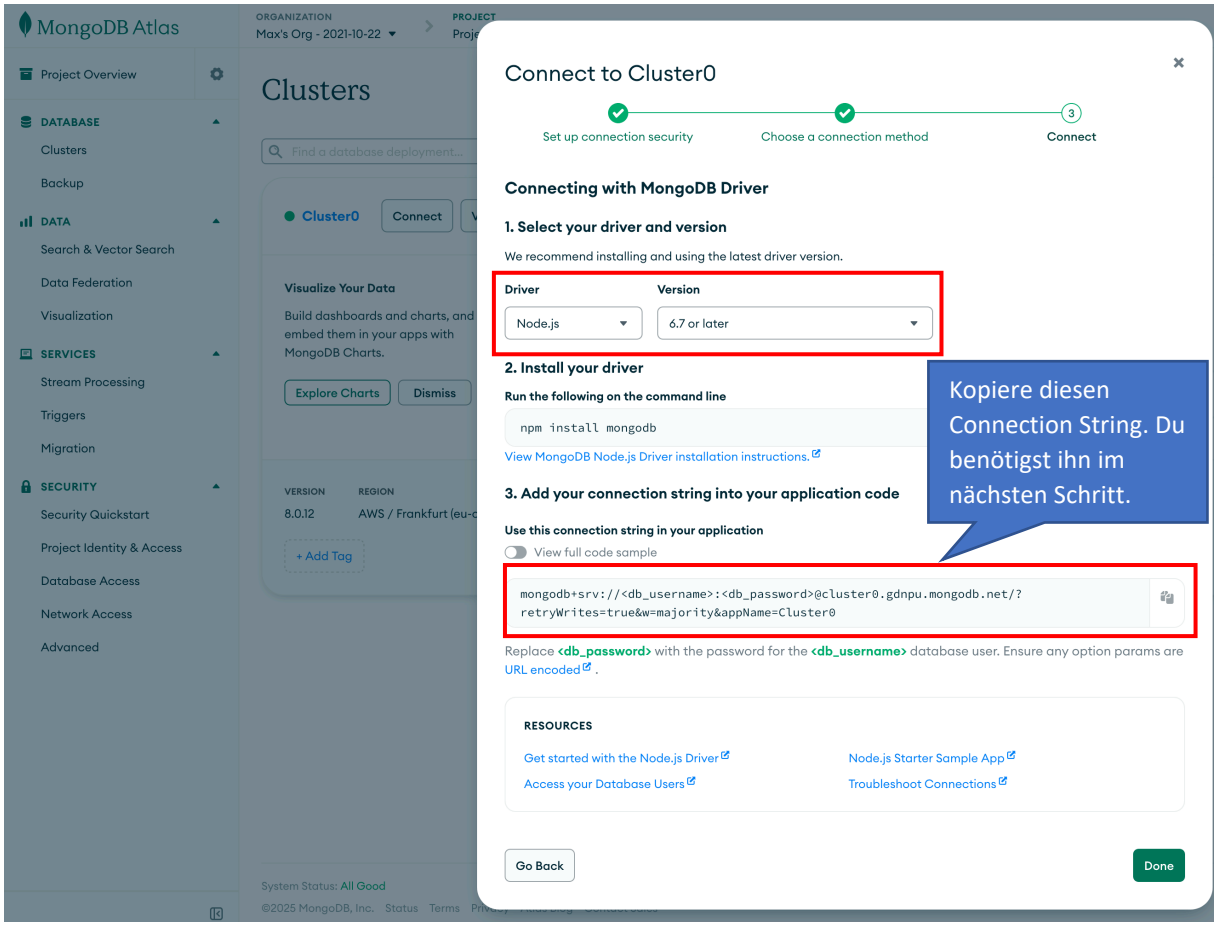

Füge dann via *Insert Document* Demodaten in die Collection ein. (Du findest die Beispieldaten auf Moodle.) Du kannst die Demodaten natürlich anpassen oder komplett selbst erzeugen (z.B. andere Filme etc.)



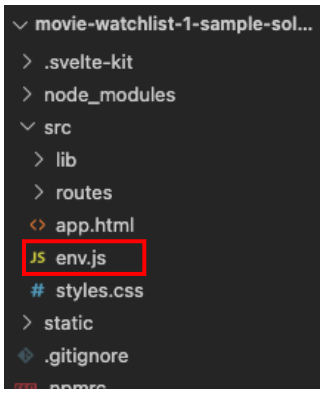
b) Um dich später in der Server-Anwendung auf die MongoDB zu verbinden, benötigst du den Connection String. Wähle dazu im Menü links den Eintrag *Database* → *Clusters* und klicke dann auf *Connect*.



c) Lade die Beispiellösung der Übung “Movie Watchlist – Teil 1” von Moodle und öffne sie in Visual Studio Code. (Oder verwende deine eigene Lösung der Übung 5.)

Danach musst du die Umgebungsvariablen einrichten, die du für den Zugriff auf deine MongoDB benötigst. Erstelle dazu im Ordner *src* eine Datei mit dem Namen *env.js:*




Füge folgenden Inhalt in env.js ein:

```
import { defineEnvVars } from '@sveltejs/kit/env';
export const variables = defineEnvVars({
    DB_URI: {}
});
```

Erstelle dann im obersten Ordner des Projekts (= der Ordner, in dem sich *package.json* befindet) eine Datei mit dem Namen ***.env*** (beachte den **Punkt** am Beginn des Dateinamens) und füge dort den Connection String **ohne Anführungszeichen** aus dem vorherigen Schritt ein:

```
DB_URI=mongodb+srv://...
```

Ersetze im Connection String `<db_username>` und `<db_password>` durch deinen DB-Benutzernamen und das dazugehörige DB-Passwort (**ohne die spitzen Klammern `<...>`**).

**Hinweis**: Verwende nicht deine MongoDB-Accountdaten, sondern die Zugangsdaten eines Datenbankusers. Wenn du dir nicht mehr sicher bist, kannst du auf MongoDB Atlas unter *Security → Database &Network Access* die Liste der DB-User anschauen und verwalten.

Lade ausserdem die Datei *db.js* von Moodle und speichere sie im Ordner *lib*. In dieser Datei sind bereits die Funktionen für den Zugriff auf die MongoDB implementiert, die du benötigst.

d) Falls du die Beispiellösung verwendest, denke daran, dass du die Abhängigkeiten einmalig mit

```
npm install
```

installieren musst.

Du benötigst ausserdem das mongodb-Package. Installiere es mit

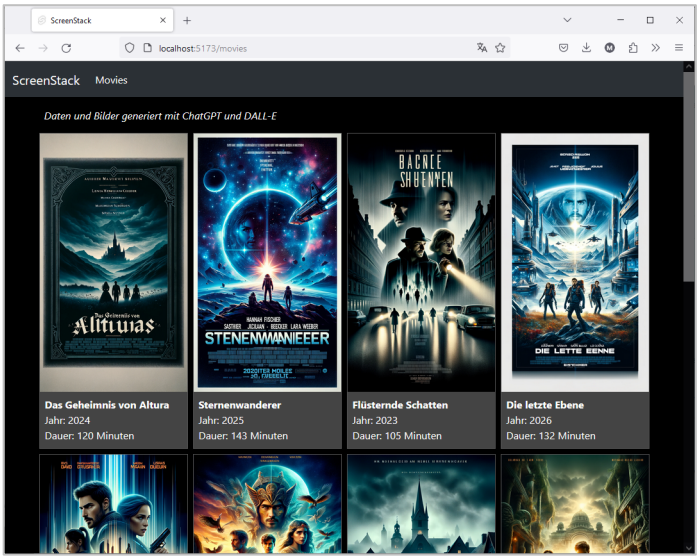
```
npm install mongodb
```

(Die beiden `npm install` Befehle musst du nur einmalig ausführen.)

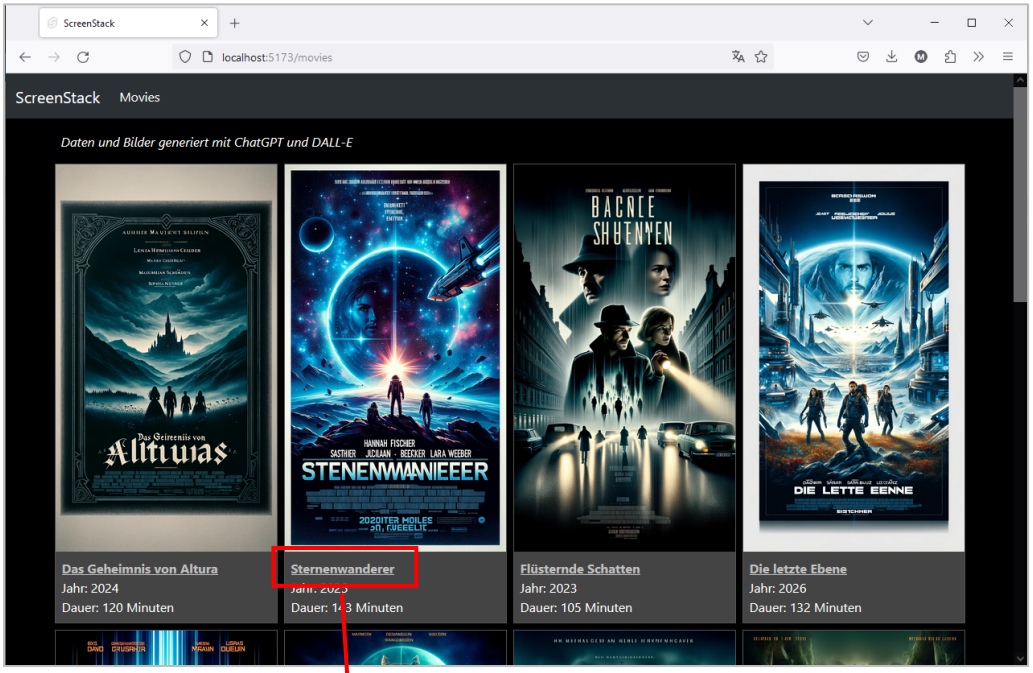
Starte die App dann wie gewohnt mit

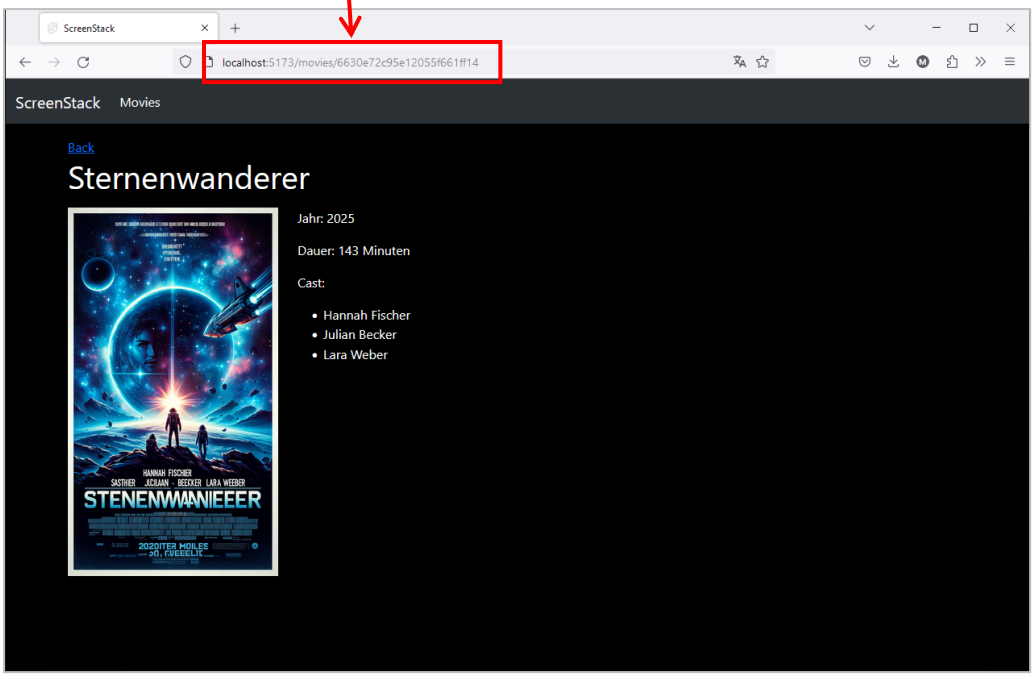
```
npm run dev
```

e) Verwende nun – wie in der Vorlesung gezeigt – die `load`-Funktion, um die Daten aus der MongoDB zu laden. Überprüfe, ob die Daten auf der Page /movies korrekt angezeigt werden:



f) Nutze die in der Vorlesung kennengelernten Konzepte, um eine Detailansicht für Filme zu implementieren. Bei einem Klick auf einen Filmtitel soll zur Detailseite navigiert werden:



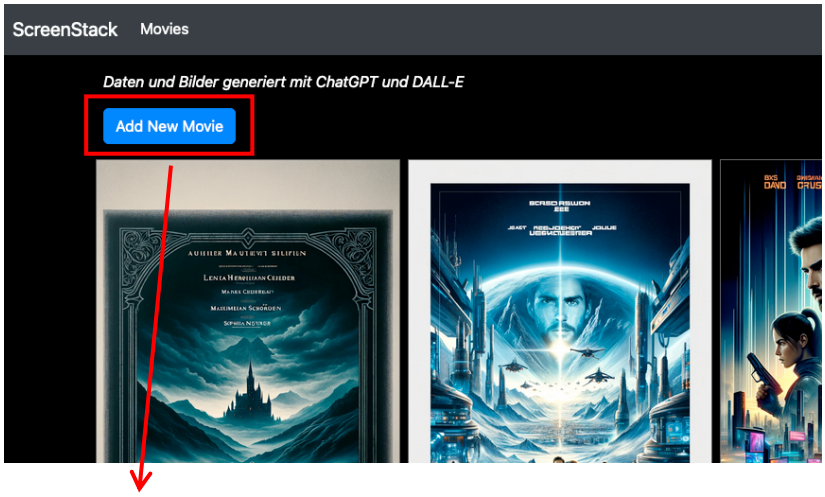




**Hinweise**:

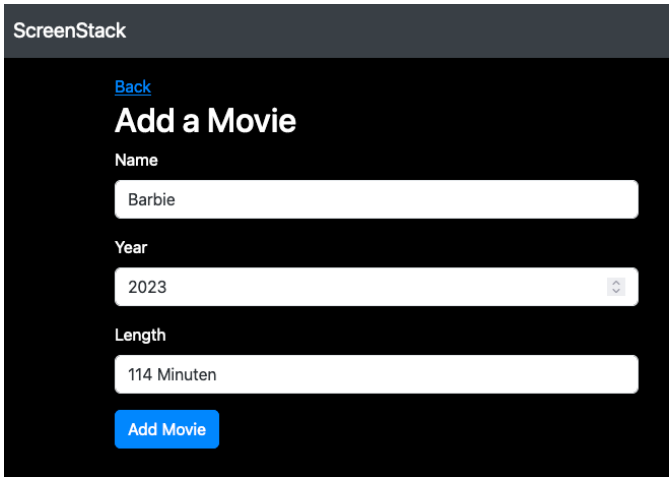
- Verwende die ID des Movies als Pfadparameter.
- Nutze in der Load-Funktion der Detailseite die in *db.js* implementierte Funktion `getMovie(id)`, um die Filminformationen zu laden.
- Die Seite muss nicht exakt so wie in den Screenshots aussehen. Du kannst beliebige Anpassungen vornehmen.

g) Erstelle eine Seite mit einem Formular, um neue Filme zu erfassen. Orientiere dich am in der Vorlesung gezeigte Beispiel.

**/movies**




**/movies/create**




**Hinweise**:

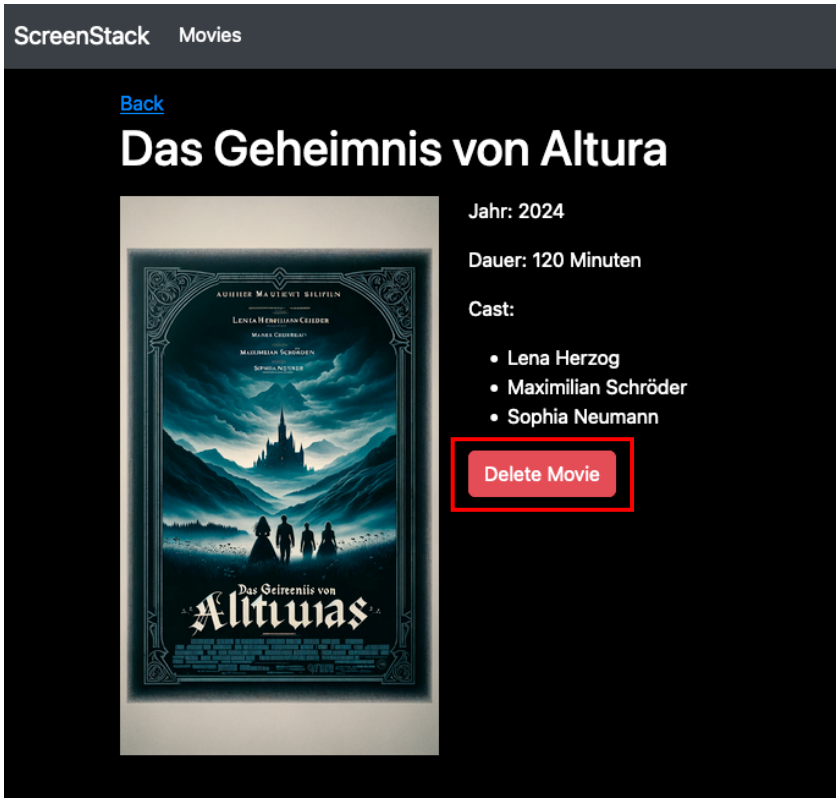
- Beschränke dich beim Erfassen von Filmen auf Namen, Jahr und Dauer. Für das Bild soll lediglich die URL erfasst werden. Du musst keinen Bild-Upload implementieren. Die Datenbankfunktion `createMovie(…)` ergänzt die fehlenden Angaben mit leeren Daten bzw. einem Platzhalterbild.

  **Beispiel** für den Aufruf der Funktion:

  ```
  let movie = {
    title: "Barbie",
    year: 2023,
    length: "114 Minuten"
  };
  await db.createMovie(movie);
  ```

- Das oben erwähnte Platzhalterbild muss den Namen *placeholder.jpg* haben und sich im Ordner */static/images* befinden. Du kannst entweder ein eigenes Platzhalterbild erstellen oder das auf Moodle zur Verfügung gestellte Bild verwenden.

h) **Optional**: Implementiere auf der Detailseite die Möglichkeit, einen Film aus der Datenbank zu löschen.




**Hinweis**: Nach dem Löschen des Films soll automatisch zur Übersichtsseite navigiert werden. Dies ist mit einem *redirect* möglich:

```
import { redirect } from "@sveltejs/kit";
// ...
export const actions = {
  delete: async ({ request }) => {
    // ...
    redirect(303, "/movies");
  },
};
```

## Ausblick

In der nächsten Woche werden wir in der Grossklasse einige optionale Erweiterungen implementieren, z.B. Movies aktualisieren oder Movies einer Watchlist hinzufügen.

## Abgabe

Gib dein Projekt als Zip-Datei auf Moodle ab. Achte darauf, dass der Ordner *node_modules* **nicht** in der Zip-Datei enthalten ist.